



Context:

I am solving an image assignment. I manually selected two corresponding landmarks in an original image and a transformed image. Using these landmarks, I computed:

- rotation angle
- scaling factor
- translation offset

I then built an affine transformation matrix, inverted it, and used OpenCV `warpAffine()` to register the transformed image back to the original image. I have attached:

- the original image-
- the transformed image-
- the computed result

and I printed values in console logs with all computed values and debugging outputs.

My Important observations:

- $M(A1) = A2$ exactly
- $M(B1) = B2$ exactly
- $M(HP1) = \text{midpoint}(A2, B2)$ exactly
- The affine matrix appears mathematically consistent

Action: Please analyze the images and logs and identify possible reasons why the final image still does not visually match the original image perfectly and it is slightly moved .

Logs:

```
/Users/klimek/.local/bin/uv run
```

```
/Users/klimek/repos/fhwn/IPAV/assignment2_ipav/.venv/bin/python
```

```
/Users/klimek/repos/fhwn/IPAV/assignment2_ipav/src/main.py
```

```
HP1 = [255. 190.]
```

```
HP2 = [256.14039649 218.80305492]
```

```
offset = [ 1.14039649 28.80305492]
```

```
HP2 - HP1 = [ 1.14039649 28.80305492]
```

```
A3 = [236.14039649 73.80305492]
```

```

B3 = [276.14039649 363.80305492]
Rotation angle: -54.20 degrees
Scaling factor: 0.95
Offset vector: [ 1.14039649 28.80305492]
rotation = -54.19571767013161
scale = 0.9474217300947747
offset = [ 1.14039649 28.80305492]
[[ 0.55425904  0.76837806 -39.82788798]
 [ -0.76837806  0.55425904 250.62718786]
 [ 0.      0.      1.    ]]
[125. 95. 1.]
[370. 225. 1.]
M(A1) = [125. 95. 1.], A2 = [125, 95]
M(B1) = [370. 225. 1.], B2 = [370, 225]
M(HP1) = [247.5 160. 1.]
HP transformed = [247.5 160. ]
2026-05-30 18:05:41.920 python[91456:6796438] +[IMKClient
subclass]: chose IMKClient_Modern
2026-05-30 18:05:41.920 python[91456:6796438] +[IMKInputSession
subclass]: chose IMKInputSession_Modern
(388, 388, 3)
(388, 388, 3)
Process finished with exit code 0

```

Show less

18:06   

Looking at your images and logs, I can identify several likely causes for the imperfect registration.

Root Cause Analysis

1. Only 2 Landmark Pairs — Underdetermined for a Full Affine Transform

A full affine transformation has **6 degrees of freedom** (rotation, scale, shear, 2D translation). You're using only 2 point correspondences (4 equations), which means:

- You're **assuming no shear** and **isotropic scaling** to close the system
- If the actual transformation includes any shear or anisotropic scaling (different scale in x vs y), your matrix won't capture it
- The "mathematically consistent" check ($M(A1)=A2$, $M(B1)=B2$) only confirms your 2 constraints are satisfied — it doesn't validate the full transform

Fix: Use at least **3 non-collinear point pairs** and solve with

`cv2.getAffineTransform()` directly, which will use all 6 DOF.

2. The Midpoint Check Is Not an Independent Validation

```
M(HP1) = midpoint(A2, B2) = (247.5, 160)
```

This holds **by linearity of affine maps** — it's guaranteed to be true for any affine matrix. It's not an independent verification that your transform is correct.

3. Small but Non-Trivial Translation Offset

```
offset = [1.14, 28.80] pixels
```

The ~29px vertical shift is significant (~7.5% of your 388px image). This is the dominant visible misalignment. Since you derived translation from the midpoint of your two landmarks, any error in landmark placement propagates directly here.

4. Scale Factor Is Applied Isotropically

```
scale = 0.947
```

You computed a single scalar scale from the ratio of distances between the two landmark pairs. But the actual transform might have **different x/y scaling**. With only 2 points you cannot detect this.

5. Landmark Selection Precision

Manual pixel-level landmark selection introduces rounding error. With a ~54° rotation, even a 1-2px error in landmark placement creates a compounded rotational error across the full image extent. At the far corners (~275px from center), a 1° rotation error = ~5px displacement.

Recommended Fix

```
python
```

```
# Use 3 point pairs instead of 2
src_pts = np.float32([[x1,y1], [x2,y2], [x3,y3]])
dst_pts = np.float32([[x1',y1'], [x2',y2'], [x3',y3']])

M = cv2.getAffineTransform(src_pts, dst_pts) # exact 6-DOF solve
result = cv2.warpAffine(transformed, M, (w, h))
```

Or if you want a least-squares fit over more points:

```
python

M, inliers = cv2.estimateAffine2D(src_pts, dst_pts) # robust, overdetermined
```

Summary Table

Issue	Impact	Severity
Only 2 landmarks (underdetermined)	Can't recover shear / aniso scale	High
~29px translation offset	Direct visible shift	High
Isotropic scale assumption	Residual stretch/compression	Medium
Manual landmark precision	Rotational error at image edges	Medium
Midpoint check not independent	False confidence in validation	Low

The core issue is that **2 points constrain 4 values but a full affine needs 6** — you're implicitly assuming a rigid + uniform-scale transform, which may not match the true transformation.

